

Pet Guardian

Mastering Relational and Non-Relational Database - Sprint 3

Enzo Okuizumi RM 561432

Gustavo Okada RM 563428

Lucas Barros RM 566422

Luna Guimarães RM 562290

Milton Marcelino RM 564836

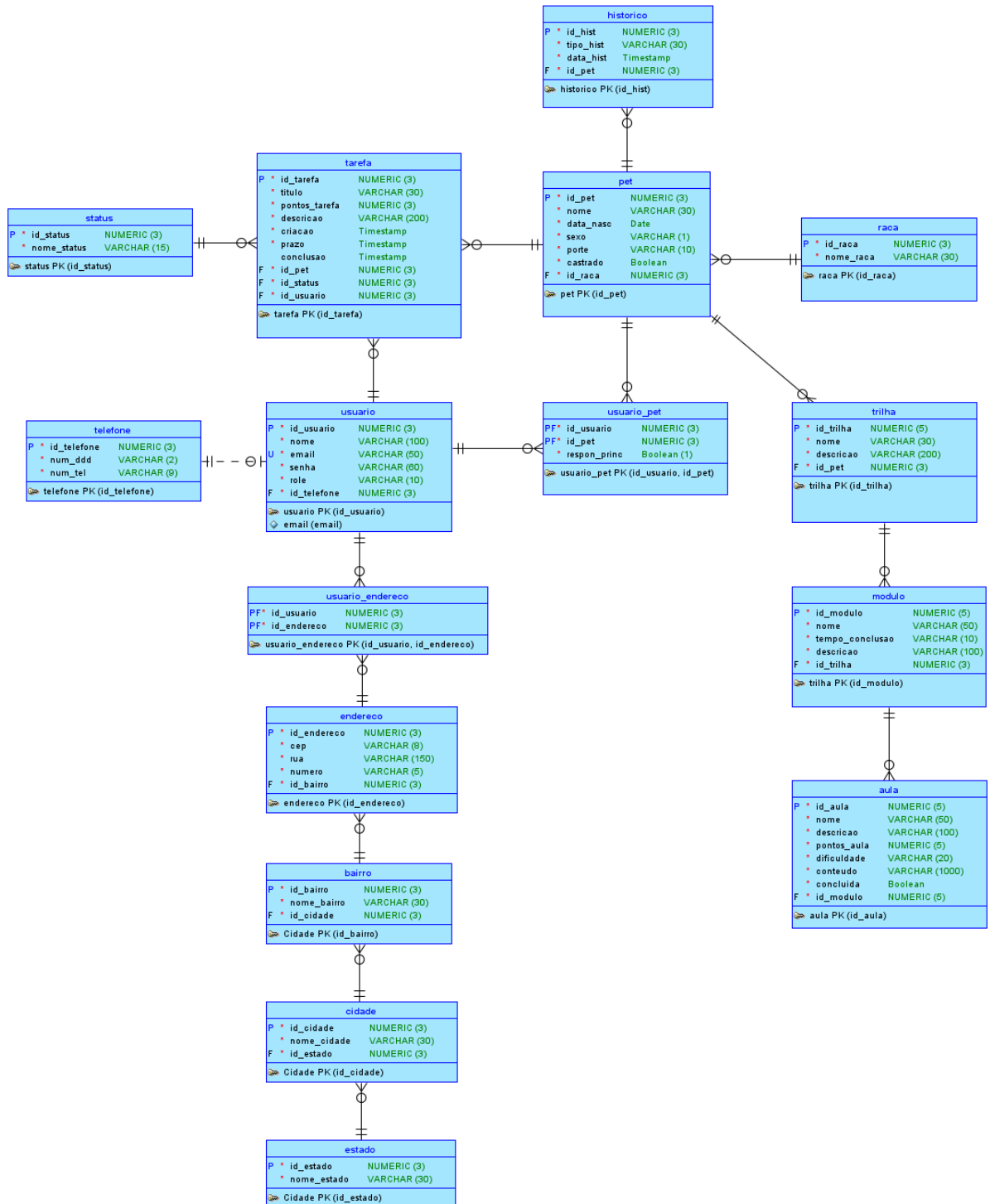
Links Importantes.....	3
Modelo Lógico.....	4
Modelo Relacional.....	5
Tabelas Criadas no Oracle SQL Developer.....	6
DML - Cargas de Dados.....	15
Objetos PL/SQL.....	17
Função 1 — Serialização JSON.....	17
Procedimento 1 — Listagem com JOIN e JSON.....	19
Função 2 — Classificação de Score.....	23
Procedimento 2 — Relatório Analítico Tabular.....	26
Trigger — Auditoria DML.....	29
Bateria de Testes.....	31
Função 1 — fn_tarefa_json.....	31
Procedimento 1 — pr_listar_tarefas_json.....	32
Função 2 — fn_classificar_pontos.....	33
Trigger — trg_audit_tarefa.....	34

Responsáveis pelo desenvolvimento: Lucas Barros / Enzo Okuizumi

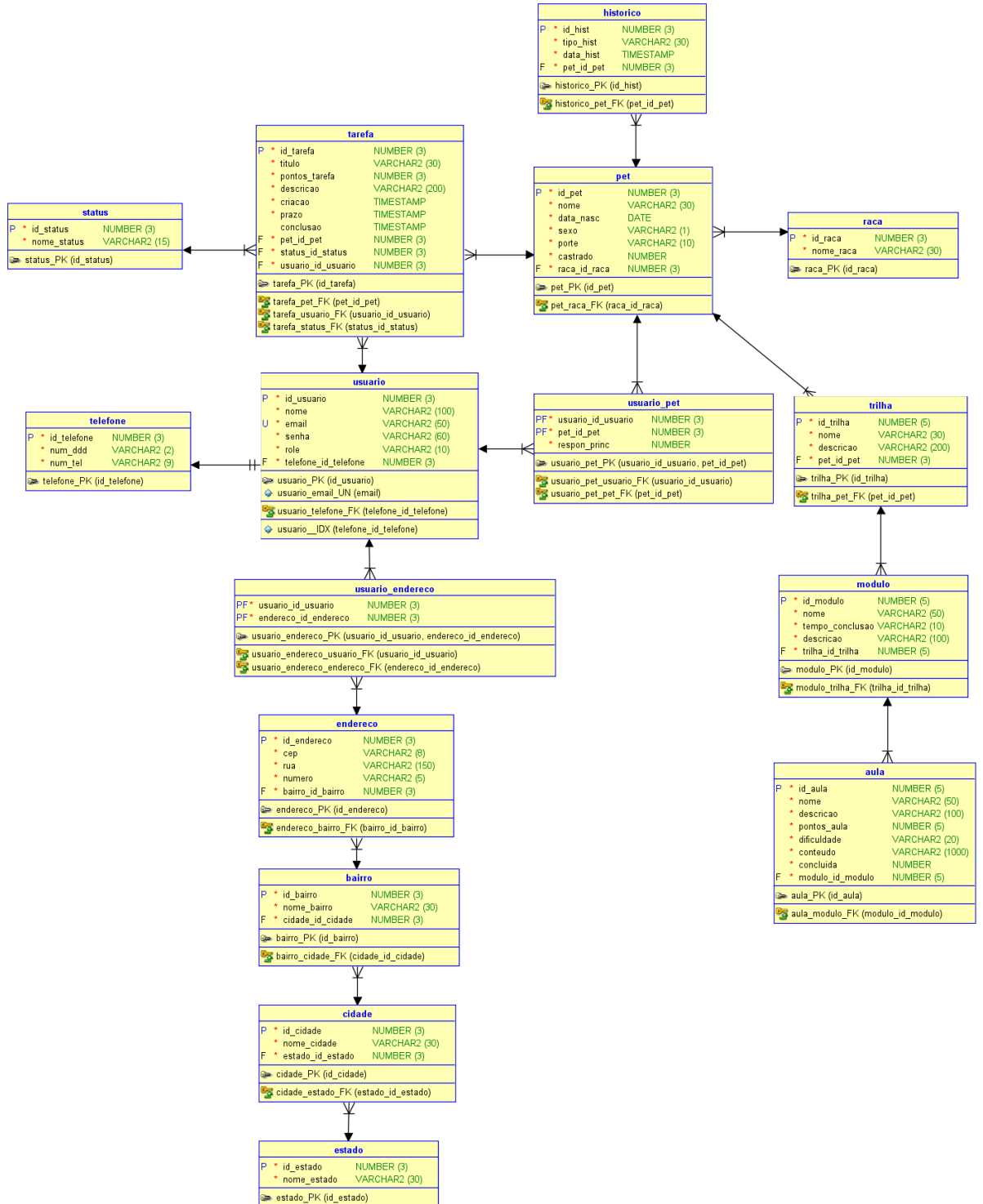
Links Importantes

Repositório Github: [Challenge-Pet-Guardian/Database-Advanced](#)

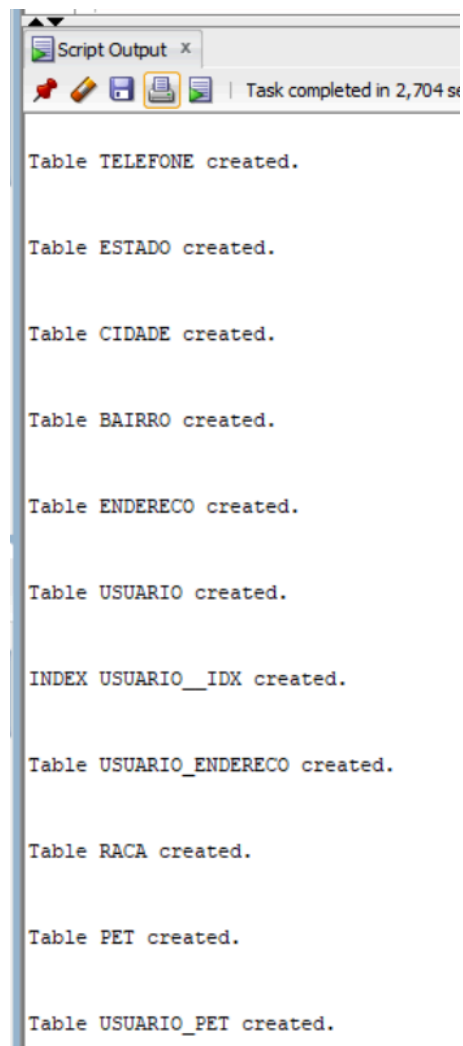
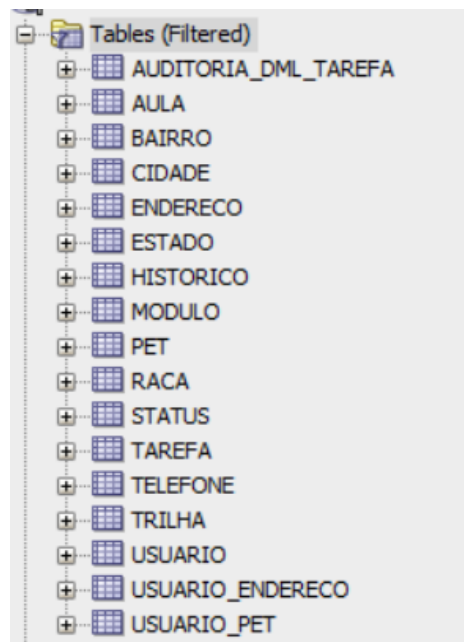
Modelo Lógico

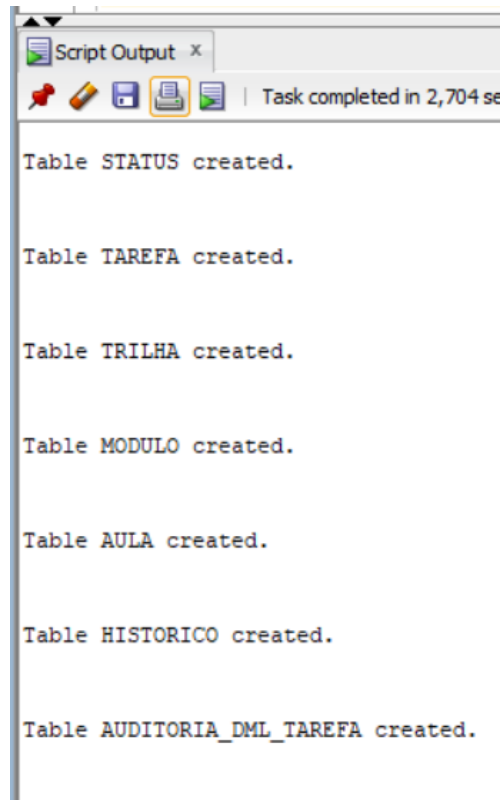


Modelo Relacional



Tabelas Criadas no Oracle SQL Developer





SQL

-- 1. TELEFONE

```
CREATE TABLE telefone (  
    id_telefone NUMBER(3) NOT NULL,  
    num_ddd     VARCHAR2(2) NOT NULL,  
    num_tel     VARCHAR2(9) NOT NULL,  
    CONSTRAINT telefone_pk PRIMARY KEY (id_telefone)  
);
```

SQL

-- 2. ESTADO

```
CREATE TABLE estado (  
    id_estado  NUMBER(3) NOT NULL,  
    nome_estado VARCHAR2(30) NOT NULL,  
    CONSTRAINT estado_pk PRIMARY KEY (id_estado)  
);
```

SQL

-- 3. CIDADE

```
CREATE TABLE cidade (  
    id_cidade          NUMBER(3) NOT NULL,  
    nome_cidade        VARCHAR2(30) NOT NULL,  
    estado_id_estado   NUMBER(3) NOT NULL,  
    CONSTRAINT cidade_pk PRIMARY KEY (id_cidade),  
    CONSTRAINT cidade_estado_fk FOREIGN KEY (estado_id_estado)  
REFERENCES estado(id_estado)  
);
```

SQL

-- 4. BAIRRO

```
CREATE TABLE bairro (  
    id_bairro          NUMBER(3) NOT NULL,  
    nome_bairro        VARCHAR2(30) NOT NULL,  
    cidade_id_cidade   NUMBER(3) NOT NULL,  
    CONSTRAINT bairro_pk PRIMARY KEY (id_bairro),  
    CONSTRAINT bairro_cidade_fk FOREIGN KEY (cidade_id_cidade)  
REFERENCES cidade(id_cidade)  
);
```

SQL

-- 5. ENDEREÇO

```
CREATE TABLE endereco (  
    id_endereco        NUMBER(3) NOT NULL,  
    cep                VARCHAR2(8) NOT NULL,  
    rua                VARCHAR2(150) NOT NULL,  
    numero             VARCHAR2(5) NOT NULL,  
    bairro_id_bairro   NUMBER(3) NOT NULL,  
    CONSTRAINT endereco_pk PRIMARY KEY (id_endereco),  
    CONSTRAINT endereco_bairro_fk FOREIGN KEY (bairro_id_bairro)  
REFERENCES bairro(id_bairro)
```



```
);
```

SQL

-- 6. USUARIO

```
CREATE TABLE usuario (  
    id_usuario          NUMBER(3) NOT NULL,  
    nome                VARCHAR2(100) NOT NULL,  
    email               VARCHAR2(50) NOT NULL,  
    senha               VARCHAR2(60) NOT NULL,  
    role                VARCHAR2(10) NOT NULL,  
    telefone_id_telefone NUMBER(3) NOT NULL,  
    CONSTRAINT usuario_pk PRIMARY KEY (id_usuario),  
    CONSTRAINT usuario_email_un UNIQUE (email),  
    CONSTRAINT ck_usuario_role CHECK (role IN ('ADMIN', 'COMUM',  
'PREMIUM')),  
    CONSTRAINT usuario_telefone_fk FOREIGN KEY  
(telefone_id_telefone) REFERENCES telefone(id_telefone)  
);  
CREATE UNIQUE INDEX usuario__idx ON usuario(telefone_id_telefone  
ASC);
```

SQL

-- 7. USUARIO_ENDEREÇO (N:M)

```
CREATE TABLE usuario_endereco (  
    usuario_id_usuario  NUMBER(3) NOT NULL,  
    endereco_id_endereco NUMBER(3) NOT NULL,  
    CONSTRAINT usuario_endereco_pk PRIMARY KEY  
(usuario_id_usuario, endereco_id_endereco),  
    CONSTRAINT usuario_endereco_usuario_fk FOREIGN KEY  
(usuario_id_usuario) REFERENCES usuario(id_usuario),  
    CONSTRAINT usuario_endereco_endereco_fk FOREIGN KEY  
(endereco_id_endereco) REFERENCES endereco(id_endereco)  
);
```

SQL

-- 8. RACA

```
CREATE TABLE raca (  
    id_raca    NUMBER(3) NOT NULL,  
    nome_raca  VARCHAR2(30) NOT NULL,  
    CONSTRAINT raca_pk PRIMARY KEY (id_raca)  
);
```

SQL

-- 9. PET

```
CREATE TABLE pet (  
    id_pet      NUMBER(3) NOT NULL,  
    nome        VARCHAR2(30) NOT NULL,  
    data_nasc   DATE NOT NULL,  
    sexo        VARCHAR2(1) NOT NULL,  
    porte       VARCHAR2(10) NOT NULL,  
    castrado    NUMBER NOT NULL,  
    raca_id_raca NUMBER(3) NOT NULL,  
    CONSTRAINT pet_pk PRIMARY KEY (id_pet),  
    CONSTRAINT ck_pet_sexo CHECK (sexo IN ('F', 'M')),  
    CONSTRAINT ck_pet_porte CHECK (porte IN ('GRANDE',  
'MEDIO', 'PEQUENO')),  
    CONSTRAINT ck_pet_castrado CHECK (castrado IN (0, 1)),  
    CONSTRAINT pet_raca_fk FOREIGN KEY (raca_id_raca) REFERENCES  
    raca(id_raca)  
);
```

SQL

-- 10. USUARIO_PET (Care Circle N:M)

```
CREATE TABLE usuario_pet (  
    usuario_id_usuario NUMBER(3) NOT NULL,
```

```

    pet_id_pet          NUMBER(3) NOT NULL,
    respon_princ        NUMBER NOT NULL,
    CONSTRAINT usuario_pet_pk    PRIMARY KEY (usuario_id_usuario,
pet_id_pet),
    CONSTRAINT ck_usuario_pet_princ CHECK (respon_princ IN (0,
1)),
    CONSTRAINT usuario_pet_usuario_fk FOREIGN KEY
(usuario_id_usuario) REFERENCES usuario(id_usuario),
    CONSTRAINT usuario_pet_pet_fk    FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet)
);

```

SQL

```

-- 11. STATUS (Domínio Fechado - 3 estados)
CREATE TABLE status (
    id_status    NUMBER(3) NOT NULL,
    nome_status  VARCHAR2(15) NOT NULL,
    CONSTRAINT status_pk PRIMARY KEY (id_status),
    CONSTRAINT ck_status_nome CHECK (nome_status IN ('CONCLUIDO',
'EXPIRADO', 'PENDENTE'))
);

```

SQL

```

-- 12. TAREFA (Tabela de Fatos de Rotina Gamificada)
CREATE TABLE tarefa (
    id_tarefa      NUMBER(3) NOT NULL,
    titulo         VARCHAR2(30) NOT NULL,
    pontos_tarefa  NUMBER(3) NOT NULL,
    descricao      VARCHAR2(200) NOT NULL,
    criacao        TIMESTAMP NOT NULL,
    prazo          TIMESTAMP NOT NULL,
    conclusao      TIMESTAMP,
    pet_id_pet     NUMBER(3) NOT NULL,

```

```

        status_id_status    NUMBER(3) NOT NULL,
        usuario_id_usuario  NUMBER(3) NOT NULL,
        CONSTRAINT tarefa_pk      PRIMARY KEY (id_tarefa),
        CONSTRAINT tarefa_pet_fk   FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet),
        CONSTRAINT tarefa_status_fk FOREIGN KEY (status_id_status)
REFERENCES status(id_status),
        CONSTRAINT tarefa_usuario_fk FOREIGN KEY (usuario_id_usuario)
REFERENCES usuario(id_usuario)
);

```

SQL

-- 13. TRILHA

```

CREATE TABLE trilha (
    id_trilha    NUMBER(5) NOT NULL,
    nome         VARCHAR2(30) NOT NULL,
    descricao    VARCHAR2(200) NOT NULL,
    pet_id_pet   NUMBER(3) NOT NULL,
    CONSTRAINT trilha_pk      PRIMARY KEY (id_trilha),
    CONSTRAINT trilha_pet_fk  FOREIGN KEY (pet_id_pet) REFERENCES
pet(id_pet)
);

```

SQL

-- 14. MODULO

```

CREATE TABLE modulo (
    id_modulo     NUMBER(5) NOT NULL,
    nome          VARCHAR2(50) NOT NULL,
    tempo_conclusao VARCHAR2(10) NOT NULL,
    descricao     VARCHAR2(100) NOT NULL,
    trilha_id_trilha NUMBER(5) NOT NULL,
    CONSTRAINT modulo_pk      PRIMARY KEY (id_modulo),

```

```
        CONSTRAINT modulo_trilha_fk FOREIGN KEY (trilha_id_trilha)
REFERENCES trilha(id_trilha)
);
```

SQL

-- 15. AULA

```
CREATE TABLE aula (
    id_aula          NUMBER(5) NOT NULL,
    nome             VARCHAR2(50) NOT NULL,
    descricao        VARCHAR2(100) NOT NULL,
    pontos_aula      NUMBER(5) NOT NULL,
    dificuldade      VARCHAR2(20) NOT NULL,
    conteudo         VARCHAR2(1000) NOT NULL,
    concluida        NUMBER NOT NULL,
    modulo_id_modulo NUMBER(5) NOT NULL,
    CONSTRAINT aula_pk          PRIMARY KEY (id_aula),
    CONSTRAINT ck_aula_concluida CHECK (concluida IN (0, 1)),
    CONSTRAINT aula_modulo_fk   FOREIGN KEY (modulo_id_modulo)
REFERENCES modulo(id_modulo)
);
```

SQL

-- 16. HISTORICO

```
CREATE TABLE historico (
    id_hist    NUMBER(3) NOT NULL,
    tipo_hist  VARCHAR2(30) NOT NULL,
    data_hist  TIMESTAMP NOT NULL,
    pet_id_pet NUMBER(3) NOT NULL,
    CONSTRAINT historico_pk          PRIMARY KEY (id_hist),
    CONSTRAINT historico_pet_fk      FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet)
);
```

SQL

```
-- TABELA DE AUDITORIA DML
CREATE TABLE auditoria_dml_tarefa (
    id_auditoria      NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
    nome_usuario      VARCHAR2(50) NOT NULL,
    tipo_operacao      VARCHAR2(10) NOT NULL,
    data_hora_operacao TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    valores_anteriores VARCHAR2(4000),
    valores_novos      VARCHAR2(4000)
);
```

DML - Cargas de Dados

```
Enzo
Worksheet Query Builder

-- 1. TELEFONE (5+ registros)
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (1, '11', '981234567');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (2, '11', '972345678');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (3, '11', '963456789');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (4, '11', '954567890');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (5, '11', '945678901');

-- 2. ESTADO (5+ registros)
INSERT INTO estado (id_estado, nome_estado) VALUES (1, 'São Paulo');
INSERT INTO estado (id_estado, nome_estado) VALUES (2, 'Rio de Janeiro');
INSERT INTO estado (id_estado, nome_estado) VALUES (3, 'Minas Gerais');
INSERT INTO estado (id_estado, nome_estado) VALUES (4, 'Paraná');
INSERT INTO estado (id_estado, nome_estado) VALUES (5, 'Santa Catarina');

-- 3. CIDADE (5+ registros)
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (1, 'São Paulo', 1);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (2, 'Campinas', 1);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (3, 'Rio de Janeiro', 2);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (4, 'Belo Horizonte', 3);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (5, 'Curitiba', 4);

-- 4. BAIRRO (5+ registros)
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (1, 'Bela Vista', 1);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (2, 'Pinheiros', 1);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (3, 'Cambui', 2);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (4, 'Copacabana', 3);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (5, 'Savassi', 4);

-- 5. ENDEREÇO (5+ registros)
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (1, '01311000', 'Avenida Paulista', '1000', 1);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (2, '05422000', 'Rua dos Pinheiros', '250', 2);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (3, '13024000', 'Rua Coronel Silva Telles', '85', 3);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (4, '22070010', 'Avenida Atlântica', '1500', 4);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (5, '30140010', 'Avenida Getúlio Vargas', '300', 5);
```

```
Enzo
Worksheet Query Builder

-- 6. USUARIO (5+ registros com senhas legíveis)
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (1, 'Carlos Silva', 'carlos.silva@email.com', 'Senha#123', 'PREMIUM', 1);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (2, 'Mariana Oliveira', 'mariana.o@email.com', 'Senha#456', 'COMUM', 2);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (3, 'Roberto Santos', 'roberto.s@email.com', 'Senha#789', 'ADMIN', 3);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (4, 'Fernanda Costa', 'fernanda.c@email.com', 'Senha#101', 'COMUM', 4);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (5, 'Lucas Mendes', 'lucas.m@email.com', 'Senha#202', 'PREMIUM', 5);

-- 7. USUARIO_ENDEREÇO (5+ registros)
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (1, 1);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (2, 2);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (3, 3);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (4, 4);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (5, 5);

-- 8. RACA (5+ registros)
INSERT INTO raca (id_raca, nome_raca) VALUES (1, 'Vira-lata (SRD)');
INSERT INTO raca (id_raca, nome_raca) VALUES (2, 'Golden Retriever');
INSERT INTO raca (id_raca, nome_raca) VALUES (3, 'Pug');
INSERT INTO raca (id_raca, nome_raca) VALUES (4, 'Gato Persa');
INSERT INTO raca (id_raca, nome_raca) VALUES (5, 'Gato Siamês');

-- 9. PET (5+ registros)
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (1, 'Thor', DATE '2021-04-10', 'M', 'GRANDE', 1, 2);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (2, 'Mel', DATE '2019-08-15', 'F', 'MEDIO', 1, 1);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (3, 'Bob', DATE '2022-01-20', 'M', 'PEQUENO', 0, 3);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (4, 'Mimi', DATE '2020-11-05', 'F', 'PEQUENO', 1, 4);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
```

```
Enzo
Worksheet Query Builder

-- 10. USUARIO_PET (Care Circle familiar - 5+ registros)
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (1, 1, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (1, 2, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (2, 3, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (3, 4, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (4, 5, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (5, 1, 0);

-- 11. STATUS (Justificativa Técnica de Domínio Fechado / Finite State Machine)
-- NOTA ARQUITETURAL: A tabela STATUS atua como uma Máquina de Estados Finita e canônica para o ciclo
-- de vida de cuidados do pet. Ela é estritamente restrita aos 3 estados discretos do sistema, validada
-- pela constraint 'ck_status_nome CHECK (nome_status IN ('CONCLUIDO', 'EXPIRADO', 'PENDENTE'))'.
-- Inserir estados adicionais violaria o domínio fechado e a integridade de negócio da aplicação.
-- Todas as demais tabelas relacionais do projeto contêm 5 ou mais registros válidos.

INSERT INTO status (id_status, nome_status) VALUES (1, 'PENDENTE');
INSERT INTO status (id_status, nome_status) VALUES (2, 'CONCLUIDO');
INSERT INTO status (id_status, nome_status) VALUES (3, 'EXPIRADO');

-- 12. TAREFA (8 registros com variação de Pet e Status)
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (1, 'Passeio Matinal', 25, 'Caminhada de 40 minutos no parque', SYSTIMESTAMP - INTERVAL '2' DAY, SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP - INTERVAL '1' DAY, NULL, 1, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (2, 'Medicamento Antibiótico', 50, 'Administrar 1 comprimido pós-refeição', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 1, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (3, 'Escovação de Pelos', 15, 'Remoção de subpelos mortos com rasqueteira', SYSTIMESTAMP - INTERVAL '3' DAY, SYSTIMESTAMP - INTERVAL '2' DAY, NULL, 1, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (4, 'Alimentação Especial', 20, 'Ração sênior com suplementação vitamínica', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 2, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (5, 'Banho e Higiene Bucal', 60, 'Banho morno com xampu neutro e escovação dental', SYSTIMESTAMP - INTERVAL '4' DAY, SYSTIMESTAMP - INTERVAL '3' DAY, SYSTIMESTAMP - INTERVAL '3' DAY, NULL, 1, 1);
```

```
Enzo
Worksheet Query Builder

-- 13. TRILHA (5+ registros)
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (1, 'Boas Maneiras Caninas', 'Comandos básicos de obediência e disciplina', 1);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (2, 'Socialização Filhotes', 'Introdução segura a outros animais e ambientes', 2);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (3, 'Controle de Ansiedade', 'Redução do estresse de separação em cães de guarda', 3);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (4, 'Enriquecimento Felino', 'Gatificação e estímulos sensoriais verticais', 4);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (5, 'Adaptação de Ambientes', 'Criação de rotinas calmas e seguras para gatos', 5);

-- 14. MODULO (5+ registros)
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (1, 'Comando Senta e Fica', '45 min', 'Técnicas de reforço positivo cc', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (2, 'Passeio Sem Puxar a Guia', '60 min', 'Equipamentos adequados e cc', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (3, 'Apresentação a Novos Cães', '30 min', 'Leitura da linguagem corpo', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (4, 'Brinquedos Recheáveis', '40 min', 'Uso de kong e tapetes de lamb', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (5, 'Instalação de Nichos', '50 min', 'Posicionamento estratégico de p', trilha_id_trilha);

-- 15. AULA (5+ registros)
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (1, 'Introdução ao Clicker', 'Fundamentos de marcação positiva', 50, 'INICIANTE', 'Aprenda a associar o clique ao reforço primário.', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (2, 'Critério de Duração do Fica', 'Aumentando o tempo gradativamente', 80, 'INTERMEDIARIO', 'Suba os segundos antes de entregar a recompensa.', 0, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (3, 'Troca de Foco na Guia', 'Atenção ao tutor em vez de estímulos da rua', 70, 'INTERMEDIARIO', 'Exercício do U-Turn quando houver tensão na guia.', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (4, 'Receitas de Enriquecimento', 'Preparo de petiscos congelados saudáveis', 40, 'INICIANTE', 'Receita de purê de abóbora e caldo de carne congelado.', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (5, 'Rotas de Fuga e Segurança', 'Evitando que o felino se sinta acuado', 60, 'AVANÇADO', 'Planejamento de rotas circulares em cômodos integrados.', 0, 1);

-- 16. HISTORICO (5+ registros)
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (1, 'Vacina Polivalente V10', SYSTIMESTAMP - INTERVAL '90' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (2, 'Check-up Clínico Preventivo', SYSTIMESTAMP - INTERVAL '45' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (3, 'Antipulgas HexGard', SYSTIMESTAMP - INTERVAL '20' DAY, 2);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (4, 'Limpeza de Tártaro', SYSTIMESTAMP - INTERVAL '60' DAY, 3);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (5, 'Vacina Ovídenim Felino V6', SYSTIMESTAMP - INTERVAL '120' DAY, 4);
```

Output Exibindo Inserções

```
Script Output x
Task completed in 2,704 seconds

1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```


Objetos PL/SQL

Função 1 — Serialização JSON

Nome: fn_tarefa_json

Assinatura: fn_tarefa_json(p_id_tarefa IN NUMBER) RETURN VARCHAR2

Recebe o ID de uma tarefa, realiza JOIN entre TAREFA, PET, STATUS e USUARIO, e monta o documento JSON caractere a caractere — sem uso de nenhuma função built-in Oracle para JSON.

Exceções tratadas:

Exceção	Código ORA	Condição
e_id_nulo_invalido	-20001	ID nulo ou ≤ 0
NO_DATA_FOUND	-20002	Tarefa não localizada
e_json_excedente	-20003	JSON gerado excede 3800 bytes
OTHERS	-20004	Qualquer erro não previsto

SQL

```
CREATE OR REPLACE FUNCTION fn_tarefa_json (  
  p_id_tarefa IN NUMBER  
) RETURN VARCHAR2 IS  
  v_titulo      tarefa.titulo%TYPE;  
  v_pontos      tarefa.pontos_tarefa%TYPE;  
  v_descricao   tarefa.descricao%TYPE;  
  v_nome_pet    pet.nome%TYPE;  
  v_nome_status status.nome_status%TYPE;  
  v_nome_usuario usuario.nome%TYPE;  
  v_json        VARCHAR2(4000);  
  
  e_id_nulo_invalido EXCEPTION;  
  e_json_excedente   EXCEPTION;  
BEGIN  
  IF p_id_tarefa IS NULL OR p_id_tarefa <= 0 THEN  
    RAISE e_id_nulo_invalido;  
  END IF;
```

```

SELECT t.titulo, t.pontos_tarefa, t.descricao, p.nome, s.nome_status, u.nome
    INTO v_titulo, v_pontos, v_descricao, v_nome_pet, v_nome_status,
v_nome_usuario
    FROM tarefa t
    JOIN pet p    ON t.pet_id_pet = p.id_pet
    JOIN status s ON t.status_id_status = s.id_status
    JOIN usuario u ON t.usuario_id_usuario = u.id_usuario
    WHERE t.id_tarefa = p_id_tarefa;

v_json := '{' ||
    '"id_tarefa":' || p_id_tarefa || ',' ||
    '"titulo":' || REPLACE(REPLACE(v_titulo, '\', '\\'), '"', '\"') || ',' ||
    '"pontos":' || v_pontos || ',' ||
    '"descricao":' || '"' || REPLACE(REPLACE(NVL(v_descricao, ''), '\', '\\'), '"', '\"') ||
    ',' ||
    '"pet":' || REPLACE(REPLACE(v_nome_pet, '\', '\\'), '"', '\"') || ',' ||
    '"status":' || v_nome_status || ',' ||
    '"cuidador_responsavel":' || REPLACE(REPLACE(v_nome_usuario, '\', '\\'), '"', '\"') ||
    '"'" ||
    '}';

IF LENGTH(v_json) > 3800 THEN
    RAISE e_json_excedente;
END IF;

RETURN v_json;

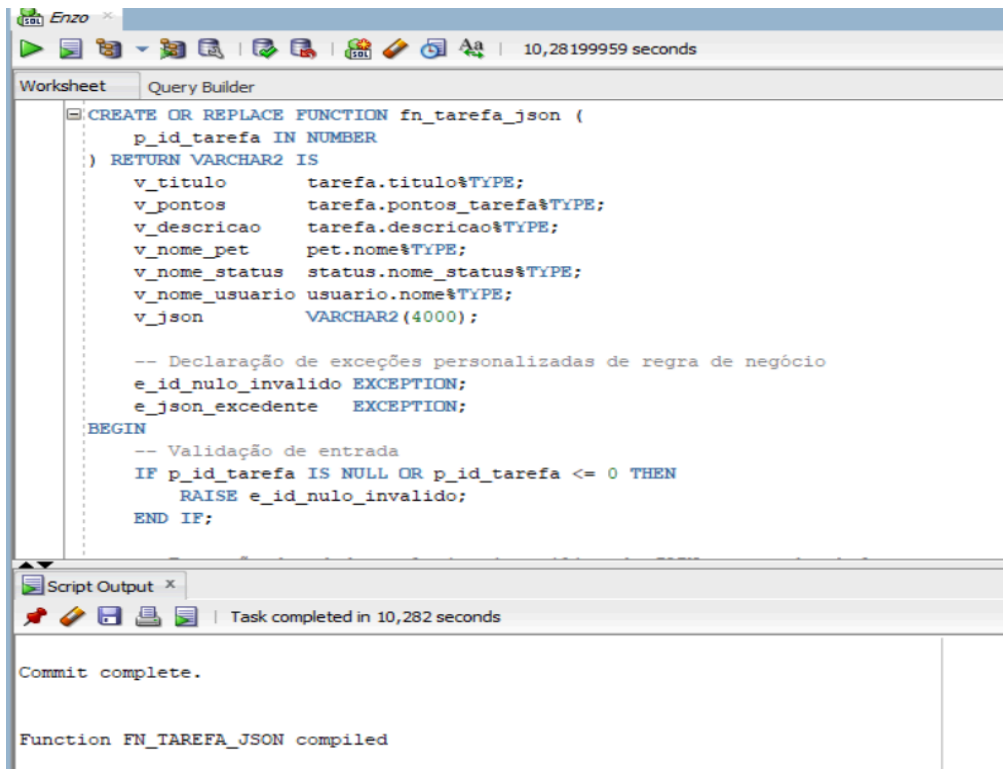
EXCEPTION
    WHEN e_id_nulo_invalido THEN
        RAISE_APPLICATION_ERROR(-20001, 'ID da tarefa fornecido é nulo ou inválido
(deve ser > 0).');
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'Nenhuma tarefa localizada com o ID
informado: ' || p_id_tarefa);
    WHEN e_json_excedente THEN
        RAISE_APPLICATION_ERROR(-20003, 'Estrutura JSON manual excedeu o limite
máximo seguro de 3800 bytes.');
```

```

    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20004, 'Falha inesperada durante a serialização
manual JSON: ' || SQLERRM);
END fn_tarefa_json;
/

```

Prints



```
CREATE OR REPLACE FUNCTION fn_tarefa_json (
  p_id_tarefa IN NUMBER
) RETURN VARCHAR2 IS
  v_titulo      tarefa.titulo%TYPE;
  v_pontos      tarefa.pontos_tarefa%TYPE;
  v_descricao   tarefa.descricao%TYPE;
  v_nome_pet    pet.nome%TYPE;
  v_nome_status status.nome_status%TYPE;
  v_nome_usuario usuario.nome%TYPE;
  v_json        VARCHAR2(4000);

  -- Declaração de exceções personalizadas de regra de negócio
  e_id_nulo_invalido EXCEPTION;
  e_json_excedente   EXCEPTION;
BEGIN
  -- Validação de entrada
  IF p_id_tarefa IS NULL OR p_id_tarefa <= 0 THEN
    RAISE e_id_nulo_invalido;
  END IF;
```

Script Output x

Task completed in 10,282 seconds

Commit complete.

Function FN_TAREFA_JSON compiled



```
SET SERVEROUTPUT ON SIZE UNLIMITED;

DECLARE
  v_resultado VARCHAR2(4000);
BEGIN
  v_resultado := fn_tarefa_json(1);
  DBMS_OUTPUT.PUT_LINE('JSON Gerado: ' || v_resultado);
END;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE(fn_tarefa_json(999));
EXCEPTION
  WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Exceção capturada: ' || SQLERRM);
END;
/
```

Script Output x

Task completed in 0,074 seconds

JSON Gerado: {"id_tarefa":1,"titulo":"Passeio Matinal","pontos":25,"descricao":"Caminhada de 40 minutos no parque","pet":"Thor","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}

PL/SQL procedure successfully completed.

Exceção capturada: ORA-20002: Nenhuma tarefa localizada com o ID informado: 999

PL/SQL procedure successfully completed.

Procedimento 1 — Listagem com JOIN e JSON

Nome: pr_listar_tarefas_json

Assinatura: pr_listar_tarefas_json(p_status_id IN NUMBER DEFAULT NULL)

Abre cursor com JOIN entre 4 tabelas, itera sobre cada tarefa e delega a serialização à fn_tarefa_json. O parâmetro p_status_id filtra por status; quando NULL, retorna todos os registros.

Exceções tratadas:

Exceção	Código ORA	Condição
e_status_inexistente	-20005	ID de status não existe na tabela
e_sem_registros	-20006	Nenhuma tarefa retornada
CURSOR_ALREADY_OPEN	-20007	Cursor já aberto em outra sessão
OTHERS	-20008	Qualquer erro não previsto

SQL

```
CREATE OR REPLACE PROCEDURE pr_listar_tarefas_json (
    p_status_id IN NUMBER DEFAULT NULL
) IS
    CURSOR c_tarefas_join IS
        SELECT t.id_tarefa, t.titulo, p.nome AS nome_pet,
            s.nome_status
        FROM tarefa t
        JOIN pet p      ON t.pet_id_pet = p.id_pet
        JOIN status s   ON t.status_id_status = s.id_status
        JOIN usuario u  ON t.usuario_id_usuario = u.id_usuario
        WHERE p_status_id IS NULL OR t.status_id_status =
            p_status_id
        ORDER BY t.id_tarefa;

    v_id_tarefa      tarefa.id_tarefa%TYPE;
    v_titulo         tarefa.titulo%TYPE;
    v_nome_pet       pet.nome%TYPE;
    v_nome_status    status.nome_status%TYPE;
    v_json_gerado    VARCHAR2(4000);
    v_total_linhas   NUMBER := 0;

    e_status_inexistente EXCEPTION;
    e_sem_registros    EXCEPTION;
    v_check_status     NUMBER;
BEGIN
    IF p_status_id IS NOT NULL THEN
```

```

        SELECT COUNT(1) INTO v_check_status FROM status WHERE
id_status = p_status_id;
        IF v_check_status = 0 THEN
            RAISE e_status_inexistente;
        END IF;
    END IF;

DBMS_OUTPUT.PUT_LINE('=====
=====');
        DBMS_OUTPUT.PUT_LINE('          RELATÓRIO DE TAREFAS
SERIALIZADAS EM JSON (MANUAL)          ');

DBMS_OUTPUT.PUT_LINE('=====
=====');

    OPEN c_tarefas_join;
    LOOP
        FETCH c_tarefas_join INTO v_id_tarefa, v_titulo,
v_nome_pet, v_nome_status;
        EXIT WHEN c_tarefas_join%NOTFOUND;

        -- Delega serialização à Função 1
        v_json_gerado := fn_tarefa_json(v_id_tarefa);
        DBMS_OUTPUT.PUT_LINE(v_json_gerado);
        v_total_linhas := v_total_linhas + 1;
    END LOOP;
    CLOSE c_tarefas_join;

    IF v_total_linhas = 0 THEN
        RAISE e_sem_registros;
    END IF;

DBMS_OUTPUT.PUT_LINE('-----
-----');
        DBMS_OUTPUT.PUT_LINE('Total de registros JSON exibidos com
sucesso: ' || v_total_linhas);

```

```

DBMS_OUTPUT.PUT_LINE('=====
=====');

EXCEPTION
    WHEN e_status_inexistente THEN
        RAISE_APPLICATION_ERROR(-20005, 'O ID de status informado
(' || p_status_id || ') não existe na tabela STATUS.');
```

```

    WHEN e_sem_registros THEN
        RAISE_APPLICATION_ERROR(-20006, 'Nenhuma tarefa atende
aos critérios de filtro informados.');
```

```

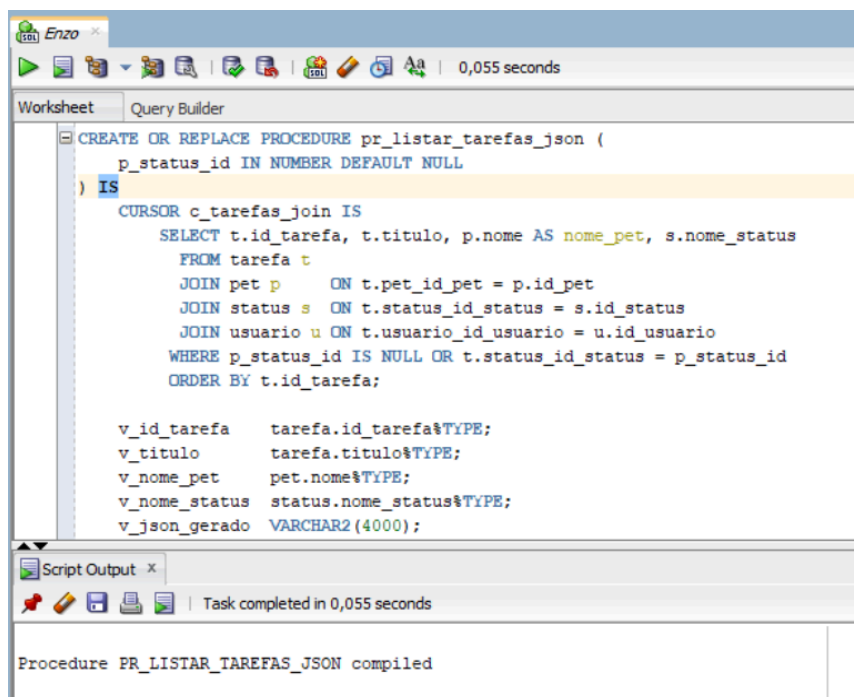
    WHEN CURSOR_ALREADY_OPEN THEN
        RAISE_APPLICATION_ERROR(-20007, 'O cursor de consulta de
tarefas já se encontra aberto em outra sessão.');
```

```

    WHEN OTHERS THEN
        IF c_tarefas_join%ISOPEN THEN CLOSE c_tarefas_join; END
IF;

        RAISE_APPLICATION_ERROR(-20008, 'Erro inesperado na
execução do Procedimento 1: ' || SQLERRM);
END pr_listar_tarefas_json;
/
```

Prints



0,116 seconds

Worksheet

Query Builder

```
BEGIN pr_listar_tarefas_json(NULL); END;

/

BEGIN
  pr_listar_tarefas_json(99);
EXCEPTION
  WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Exceção: ' || SQLERRM);
END;
```

Script Output x

Task completed in 0,116 seconds

RELATÓRIO DE TAREFAS SERIALIZADAS EM JSON (MANUAL)

[{"id_tarefa":1,"titulo":"Passeio Matinal","pontoss":25,"descricao":"Caminhada de 40 minutos no parque","pet":"Thor","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}
{ "id_tarefa":2,"titulo":"Medicamento Antibiótico","pontoss":50,"descricao":"Administrar 1 comprimido pós-refeição","pet":"Thor","status":"PENDENTE","cuidador_responsavel":"Carlos Silva"}
{ "id_tarefa":3,"titulo":"Escovação de Pelos","pontoss":15,"descricao":"Remoção de subpelos mortos com rasqueteira","pet":"Thor","status":"EXPIRADO","cuidador_responsavel":"Lucas Mendes"}
{ "id_tarefa":4,"titulo":"Alimentação Especial","pontoss":20,"descricao":"Ração sênior com suplementação vitamínica","pet":"Mel","status":"PENDENTE","cuidador_responsavel":"Carlos Silva"}
{ "id_tarefa":5,"titulo":"Banho e Higiene Bucal","pontoss":60,"descricao":"Banho morno com xampu neutro e escovação dental","pet":"Mel","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}
{ "id_tarefa":6,"titulo":"Corte de Unhas","pontoss":30,"descricao":"Aparar as pontas das unhas com alicate apropriado","pet":"Bob","status":"PENDENTE","cuidador_responsavel":"Mariana Oliveira"}
{ "id_tarefa":7,"titulo":"Limpeza da Caixa de Areia","pontoss":20,"descricao":"Higienização completa e troca da sílica","pet":"Mimi","status":"CONCLUIDO","cuidador_responsavel":"Roberto Santos"}
{ "id_tarefa":8,"titulo":"Estímulo com Varinha","pontoss":15,"descricao":"Sessão de brincadeiras de caça por 20 minutos","pet":"Luna","status":"PENDENTE","cuidador_responsavel":"Fernanda Costa"}]

Total de registros JSON exibidos com sucesso: 8

PL/SQL procedure successfully completed.

Exceção: ORA-20005: O ID de status informado (99) não existe na tabela STATUS.

PL/SQL procedure successfully completed.

Função 2 — Classificação de Score

Nome: fn_classificar_pontos

Assinatura: fn_classificar_pontos(p_pontos IN NUMBER) RETURN VARCHAR2

Implementa a regra de negócio de gamificação de bem-estar do PetGuardian, retornando a faixa de classificação do pet conforme a pontuação.

Regras de classificação:

Faixa de Pontos	Classificação
0 – 20	BRONZE (BÁSICO)
21 – 45	PRATA (INTERMEDIÁRIO)
46 – 75	OURO (AVANÇADO)
76 – 1000	DIAMANTE (MASTER)

Exceções tratadas:

Exceção	Código ORA	Condição
e_pontos_nulo	-20009	Pontuação nula
e_pontos_negativo	-20010	Pontuação < 0
e_pontos_excesso	-20011	Pontuação > 1000
OTHERS	-20012	Qualquer erro não previsto

SQL

```
CREATE OR REPLACE FUNCTION fn_classificar_pontos ( p_pontos IN
NUMBER) RETURN VARCHAR2 IS
    v_classificacao VARCHAR2(50);
    e_pontos_nulo EXCEPTION;
    e_pontos_negativo EXCEPTION;
    e_pontos_excesso EXCEPTION;
BEGIN
    IF p_pontos IS NULL THEN
        RAISE e_pontos_nulo;
    ELSIF p_pontos < 0 THEN
        RAISE e_pontos_negativo;
    ELSIF p_pontos > 1000 THEN
        RAISE e_pontos_excesso;
    END IF;

    IF p_pontos <= 20 THEN
        v_classificacao := 'BRONZE (BÁSICO)';
    ELSIF p_pontos <= 45 THEN
        v_classificacao := 'PRATA (INTERMEDIÁRIO)';
    ELSIF p_pontos <= 75 THEN
        v_classificacao := 'OURO (AVANÇADO)';
    ELSE
        v_classificacao := 'DIAMANTE (MASTER)';
    END IF;

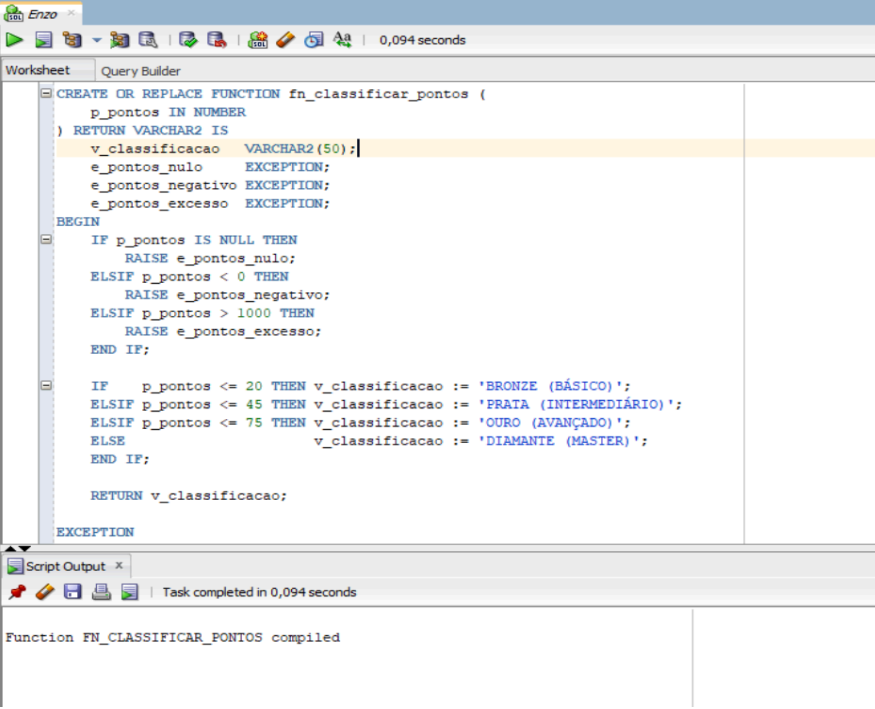
    RETURN v_classificacao;
EXCEPTION
    WHEN e_pontos_nulo THEN
        RAISE_APPLICATION_ERROR(-20009, 'Pontuação informada não
pode ser nula.');
```

```
    WHEN e_pontos_negativo THEN
        RAISE_APPLICATION_ERROR(-20010, 'A pontuação não pode ser
negativa: ' || p_pontos);
    WHEN e_pontos_excesso THEN
        RAISE_APPLICATION_ERROR(-20011, 'Pontuação acima do
limite máximo permitido de 1000 pontos: ' || p_pontos);
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20012, 'Erro ao classificar
pontos na Função 2: ' || SQLERRM);
```



```
END fn_classificar_pontos;  
/
```

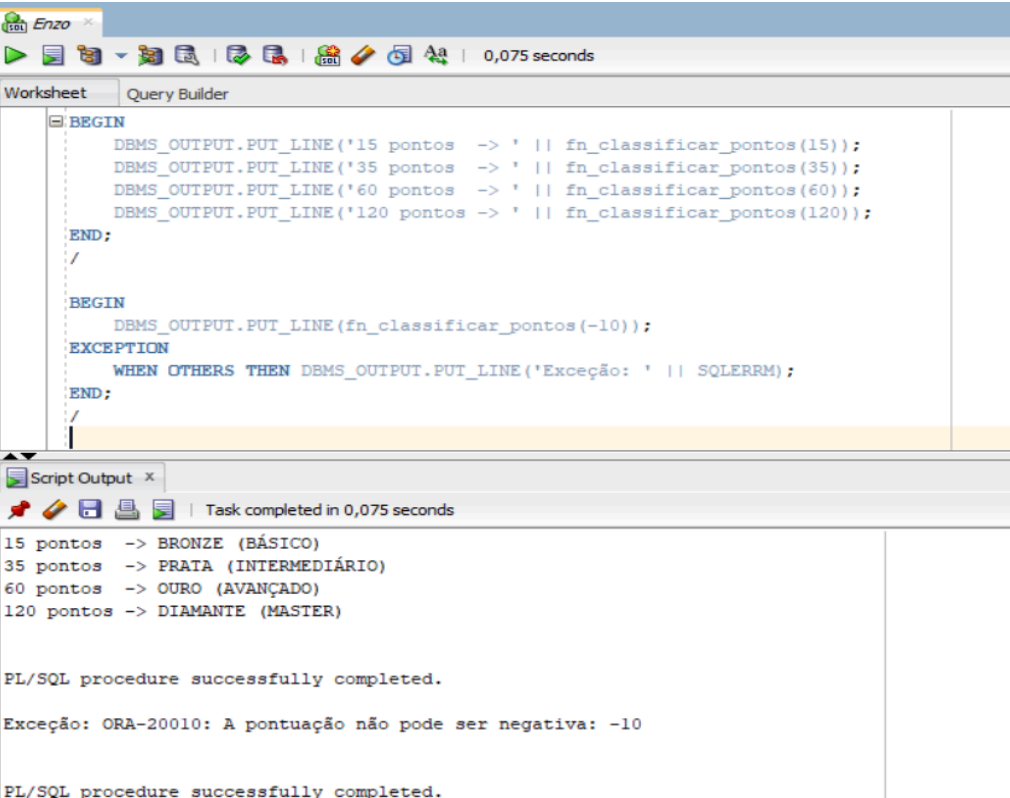
Prints



The screenshot shows the Enzo IDE interface. The top toolbar includes icons for running, saving, and other development actions. The 'Worksheet' tab is active, displaying the SQL code for creating the function. The 'Query Builder' tab is also visible. The script output at the bottom indicates that the function was compiled successfully.

```
CREATE OR REPLACE FUNCTION fn_classificar_pontos (  
  p_pontos IN NUMBER  
) RETURN VARCHAR2 IS  
  v_classificacao VARCHAR2(50);  
  e_pontos_nulo EXCEPTION;  
  e_pontos_negativo EXCEPTION;  
  e_pontos_excesso EXCEPTION;  
  
BEGIN  
  IF p_pontos IS NULL THEN  
    RAISE e_pontos_nulo;  
  ELSIF p_pontos < 0 THEN  
    RAISE e_pontos_negativo;  
  ELSIF p_pontos > 1000 THEN  
    RAISE e_pontos_excesso;  
  END IF;  
  
  IF p_pontos <= 20 THEN v_classificacao := 'BRONZE (BÁSICO)';  
  ELSIF p_pontos <= 45 THEN v_classificacao := 'PRATA (INTERMEDIÁRIO)';  
  ELSIF p_pontos <= 75 THEN v_classificacao := 'OURO (AVANÇADO)';  
  ELSE  
    v_classificacao := 'DIAMANTE (MASTER)';  
  END IF;  
  
  RETURN v_classificacao;  
  
EXCEPTION
```

Script Output x
Task completed in 0,094 seconds
Function FN_CLASSIFICAR_PONTOS compiled



The screenshot shows the Enzo IDE interface with the same toolbar. The 'Worksheet' tab displays the SQL code for testing the function. The 'Query Builder' tab is also visible. The script output at the bottom shows the results of the function calls and an error message for a negative input.

```
BEGIN  
  DBMS_OUTPUT.PUT_LINE('15 pontos -> ' || fn_classificar_pontos(15));  
  DBMS_OUTPUT.PUT_LINE('35 pontos -> ' || fn_classificar_pontos(35));  
  DBMS_OUTPUT.PUT_LINE('60 pontos -> ' || fn_classificar_pontos(60));  
  DBMS_OUTPUT.PUT_LINE('120 pontos -> ' || fn_classificar_pontos(120));  
END;  
/  
  
BEGIN  
  DBMS_OUTPUT.PUT_LINE(fn_classificar_pontos(-10));  
EXCEPTION  
  WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Exceção: ' || SQLERRM);  
END;  
/
```

Script Output x
Task completed in 0,075 seconds
15 pontos -> BRONZE (BÁSICO)
35 pontos -> PRATA (INTERMEDIÁRIO)
60 pontos -> OURO (AVANÇADO)
120 pontos -> DIAMANTE (MASTER)

PL/SQL procedure successfully completed.

Exceção: ORA-20010: A pontuação não pode ser negativa: -10

PL/SQL procedure successfully completed.

Procedimento 2 — Relatório Analítico Tabular

Nome: pr_resumo_pontos_tarefas

Parâmetros: nenhum

Gera relatório tabular de pontos de tarefas com dois níveis de agrupamento (Control Break): Pet (Cat 1) e Status (Cat 2). O somatório por combinação, subtotal por Pet e total geral são calculados via variáveis acumuladoras em PL/SQL. A formatação de colunas usa RPAD/LPAD.

Exceções tratadas:

Exceção	Código ORA	Condição
e_sem_fatos_cadastrados	-20013	Cursor retornou zero registros
VALUE_ERROR	-20014	Estouro numérico na conversão
OTHERS	-20015	Erro fatal genérico

SQL

```
CREATE OR REPLACE PROCEDURE pr_resumo_pontos_tarefas IS
    CURSOR c_fatos IS
        SELECT p.id_pet, p.nome AS nome_pet, s.id_status,
               s.nome_status, t.pontos_tarefa
        FROM tarefa t
        JOIN pet p ON t.pet_id_pet = p.id_pet
        JOIN status s ON t.status_id_status = s.id_status
        ORDER BY p.id_pet ASC, s.id_status ASC;
    r_linha c_fatos%ROWTYPE;
    v_pet_atual pet.id_pet%TYPE := NULL;
    v_nome_pet_atual pet.nome%TYPE := NULL;
    v_status_atual status.id_status%TYPE := NULL;
    v_nome_status_atual status.nome_status%TYPE := NULL;
    v_soma_combinacao NUMBER(10,2) := 0;
    v_subtotal_pet NUMBER(10,2) := 0;
    v_total_geral NUMBER(10,2) := 0;
    v_qtd_linhas NUMBER := 0;
    e_sem_fatos_cadastrados EXCEPTION;
BEGIN
    OPEN c_fatos;
    DBMS_OUTPUT.PUT_LINE(RPAD('Pet (Cat 1)', 18) || ' ' ||
                          RPAD('Status (Cat 2)', 16) || ' ' || LPAD('Pontos', 12));
```

```

        DBMS_OUTPUT.PUT_LINE(RPAD('-', 18, '-') || ' ' || RPAD('-',
16, '-') || ' ' || LPAD('-', 12, '-'));
    LOOP
        FETCH c_fatos INTO r_linha;
        IF (c_fatos%NOTFOUND OR r_linha.id_pet <> v_pet_atual OR
r_linha.id_status <> v_status_atual) AND v_pet_atual IS NOT NULL
THEN
            DBMS_OUTPUT.PUT_LINE(
                RPAD(TO_CHAR(v_pet_atual) || ' - ' ||
v_nome_pet_atual, 18) || ' ' ||
                RPAD(TO_CHAR(v_status_atual) || ' - ' ||
v_nome_status_atual, 16) || ' ' ||
                LPAD(TO_CHAR(v_soma_combinacao, 'FM999990.00'),
12)
            );
            v_subtotal_pet := v_subtotal_pet + v_soma_combinacao;
            v_total_geral := v_total_geral + v_soma_combinacao;
            v_soma_combinacao := 0;
            IF c_fatos%NOTFOUND OR r_linha.id_pet <> v_pet_atual
THEN
                DBMS_OUTPUT.PUT_LINE(RPAD('Sub Total', 35) ||
LPAD(TO_CHAR(v_subtotal_pet, 'FM999990.00'), 12));
                v_subtotal_pet := 0;
            END IF;
        END IF;
        EXIT WHEN c_fatos%NOTFOUND;
        v_pet_atual := r_linha.id_pet;
        v_nome_pet_atual := r_linha.nome_pet;
        v_status_atual := r_linha.id_status;
        v_nome_status_atual := r_linha.nome_status;
        v_soma_combinacao := v_soma_combinacao +
r_linha.pontos_tarefa;
        v_qtd_linhas := v_qtd_linhas + 1;
    END LOOP;
    CLOSE c_fatos;
    IF v_qtd_linhas = 0 THEN
        RAISE e_sem_fatos_cadastrados;
    END IF;

```

```

        DBMS_OUTPUT.PUT_LINE(RPAD('Total Geral', 35) ||
LPAD(TO_CHAR(v_total_geral, 'FM999990.00'), 12));
EXCEPTION
    WHEN e_sem_fatos_cadastrados THEN
        RAISE_APPLICATION_ERROR(-20013, 'Nenhum registro de
tarefa/fato localizado para sumarização.');
```

WHEN VALUE_ERROR THEN
 RAISE_APPLICATION_ERROR(-20014, 'Estouro de capacidade ou
erro de conversão numérica durante o somatório manual.');

WHEN OTHERS THEN
 IF c_fatos%ISOPEN THEN CLOSE c_fatos; END IF;
 RAISE_APPLICATION_ERROR(-20015, 'Erro fatal no cálculo do
Procedimento 2: ' || SQLERRM);
 END pr_resumo_pontos_tarefas;
/

Prints

The screenshot shows the Enzo SQL IDE interface. The main window displays the SQL code for creating or replacing the procedure `pr_resumo_pontos_tarefas`. The code includes a cursor `c_fatos` that selects data from the `tarefa` table, joined with `pet` and `status` tables. Below the cursor definition, there are variable declarations for `v_pet_atual`, `v_nome_pet_atual`, and `v_status_atual`, all initialized to `NULL`.

The bottom panel, titled "Script Output", shows the message "Task completed in 0,09 seconds" and "Procedure PR_RESUMO_PONTOS_TAREFAS compiled".

```

CREATE OR REPLACE PROCEDURE pr_resumo_pontos_tarefas IS
    CURSOR c_fatos IS
        SELECT p.id_pet, p.nome AS nome_pet,
               s.id_status, s.nome_status,
               t.pontos_tarefa
        FROM tarefa t
        JOIN pet p    ON t.pet_id_pet = p.id_pet
        JOIN status s ON t.status_id_status = s.id_status
        ORDER BY p.id_pet ASC, s.id_status ASC;

    r_linha c_fatos%ROWTYPE;

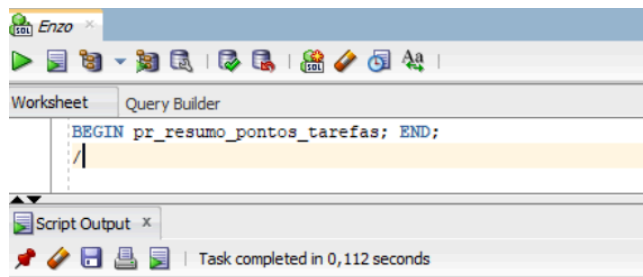
    v_pet_atual      pet.id_pet%TYPE      := NULL;
    v_nome_pet_atual pet.nome%TYPE        := NULL;
    v_status_atual   status.id_status%TYPE := NULL;

```

Script Output x

Task completed in 0,09 seconds

Procedure PR_RESUMO_PONTOS_TAREFAS compiled



Procedure PR_RESUMO_PONTOS_TAREFAS compiled

Pet (Cat 1)	Status (Cat 2)	Pontos
1 - Thor	1 - PENDENTE	50.00
1 - Thor	2 - CONCLUIDO	25.00
1 - Thor	3 - EXPIRADO	15.00
Sub Total		90.00
2 - Mel	1 - PENDENTE	20.00
2 - Mel	2 - CONCLUIDO	60.00
Sub Total		80.00
3 - Bob	1 - PENDENTE	30.00
Sub Total		30.00
4 - Mimi	2 - CONCLUIDO	20.00
Sub Total		20.00
5 - Luna	1 - PENDENTE	15.00
Sub Total		15.00
Total Geral		235.00

PL/SQL procedure successfully completed.

Trigger — Auditoria DML

Nome: trg_audit_tarefa

Evento: AFTER INSERT OR UPDATE OR DELETE ON tarefa FOR EACH ROW

Tabela de destino: auditoria_dml_tarefa

Captura automaticamente cada operação DML na tabela TAREFA, registrando: usuário de banco (USER), tipo de operação, timestamp e os valores anteriores (:OLD) e novos (:NEW) dos campos relevantes.

SQL

```
CREATE OR REPLACE TRIGGER trg_audit_tarefa
AFTER INSERT OR UPDATE OR DELETE ON tarefa
FOR EACH ROW
DECLARE
    v_tipo_operacao VARCHAR2(10);
    v_valores_old   VARCHAR2(4000) := NULL;
    v_valores_new   VARCHAR2(4000) := NULL;
BEGIN
    IF INSERTING THEN
        v_tipo_operacao := 'INSERT';
```

```

        v_valores_new := 'ID_TAREFA=' || :NEW.id_tarefa || ';'
TITULO=' || :NEW.titulo || '; PONTOS=' || :NEW.pontos_tarefa ||
'; STATUS=' || :NEW.status_id_status || '; PET=' ||
:NEW.pet_id_pet || '; USUARIO=' || :NEW.usuario_id_usuario;
        INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)
        VALUES (USER, v_tipo_operacao, SYSTIMESTAMP, NULL,
v_valores_new);
    ELSIF UPDATING THEN
        v_tipo_operacao := 'UPDATE';
        v_valores_old := 'ID_TAREFA=' || :OLD.id_tarefa || ';'
TITULO=' || :OLD.titulo || '; PONTOS=' || :OLD.pontos_tarefa ||
'; STATUS=' || :OLD.status_id_status || '; PET=' ||
:OLD.pet_id_pet || '; USUARIO=' || :OLD.usuario_id_usuario;
        v_valores_new := 'ID_TAREFA=' || :NEW.id_tarefa || ';'
TITULO=' || :NEW.titulo || '; PONTOS=' || :NEW.pontos_tarefa ||
'; STATUS=' || :NEW.status_id_status || '; PET=' ||
:NEW.pet_id_pet || '; USUARIO=' || :NEW.usuario_id_usuario;
        INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)
        VALUES (USER, v_tipo_operacao, SYSTIMESTAMP,
v_valores_old, v_valores_new);
    ELSIF DELETING THEN
        v_tipo_operacao := 'DELETE';
        v_valores_old := 'ID_TAREFA=' || :OLD.id_tarefa || ';'
TITULO=' || :OLD.titulo || '; PONTOS=' || :OLD.pontos_tarefa ||
'; STATUS=' || :OLD.status_id_status || '; PET=' ||
:OLD.pet_id_pet || '; USUARIO=' || :OLD.usuario_id_usuario;
        INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)
        VALUES (USER, v_tipo_operacao, SYSTIMESTAMP,
v_valores_old, NULL);
    END IF;
EXCEPTION
    WHEN OTHERS THEN

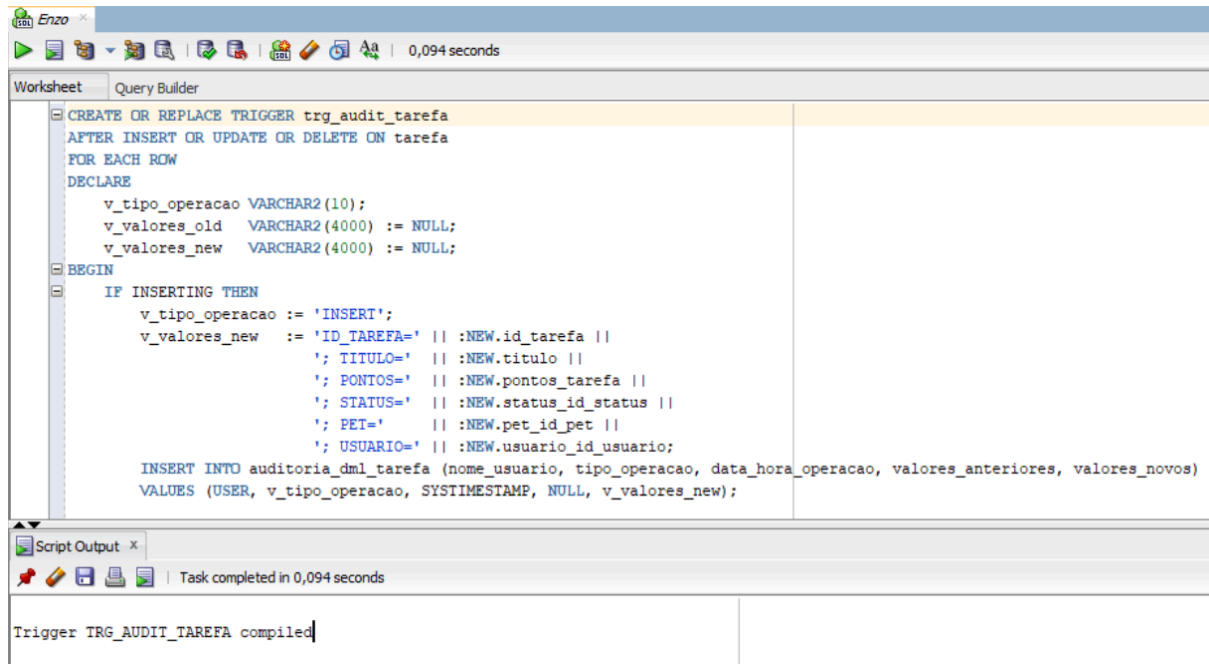
```

```

        RAISE_APPLICATION_ERROR(-20016, 'Erro fatal na execução
da trigger: ' || SQLERRM);
END trg_audit_tarefa;
/

```

Prints

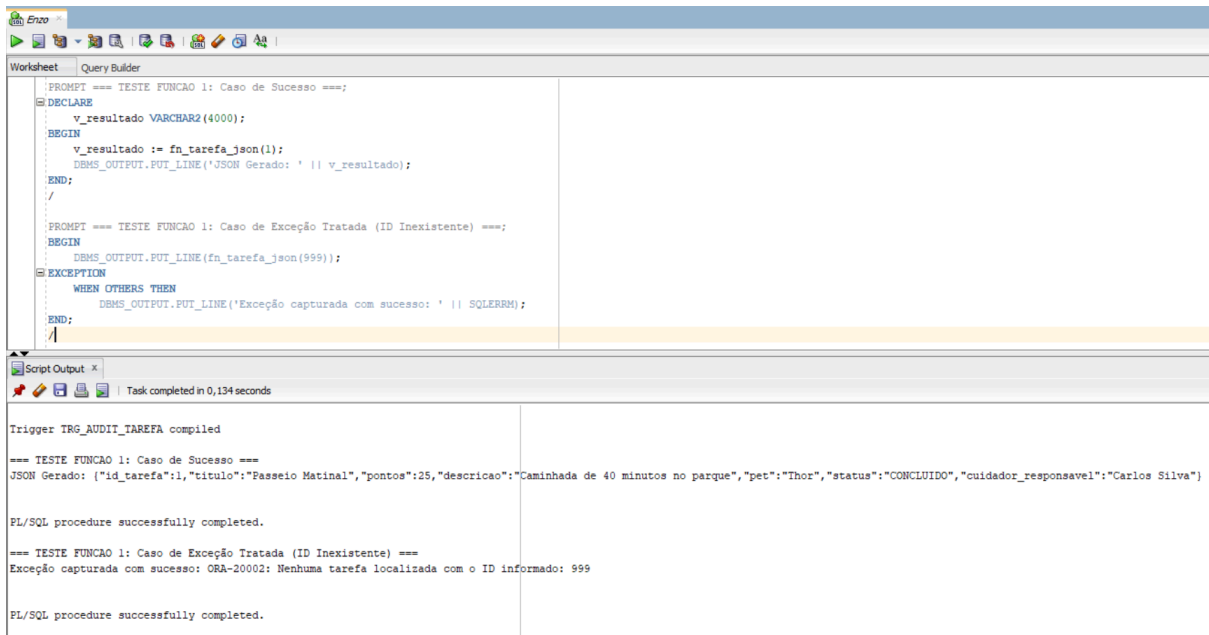


Bateria de Testes

Função 1 — fn_tarefa_json

Caso	Entrada	Resultado Esperado
Sucesso	p_id_tarefa = 1	JSON com dados da tarefa ID 1
Exceção — ID inexistente	p_id_tarefa = 999	ORA-20002

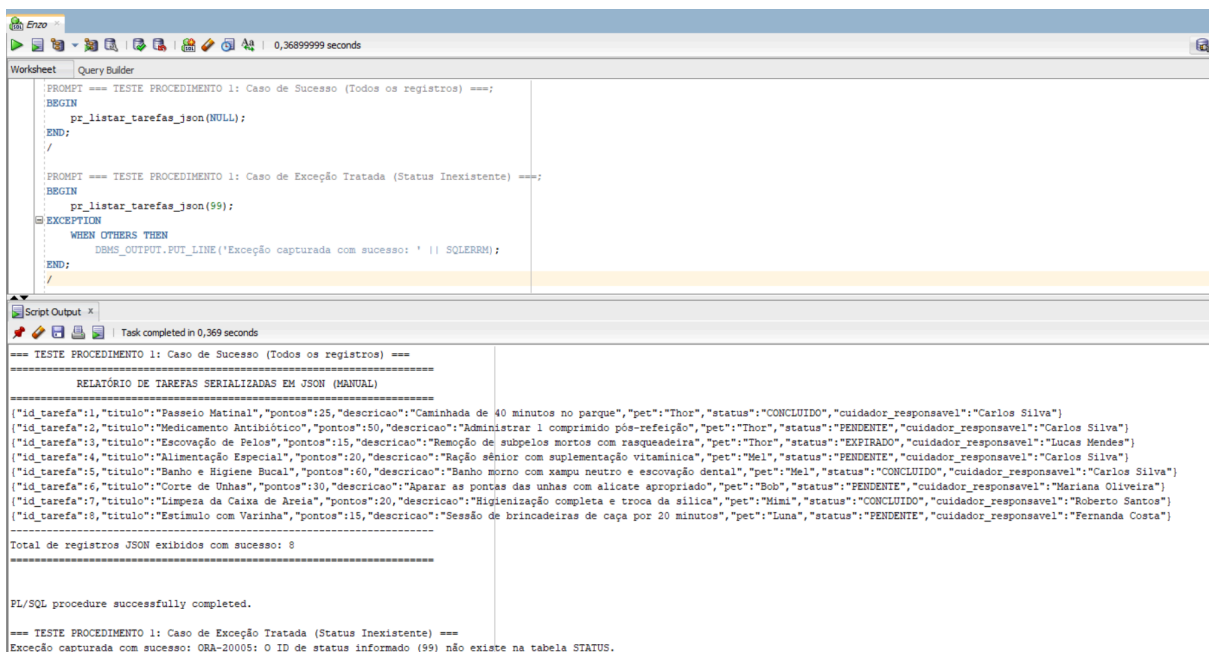
Prints



Procedimento 1 — pr_listar_tarefas_json

Caso	Entrada	Resultado Esperado
Sucesso	p_status_id = NULL	Todos os JSONs + total de registros
Exceção — status inexistente	p_status_id = 99	ORA-20005

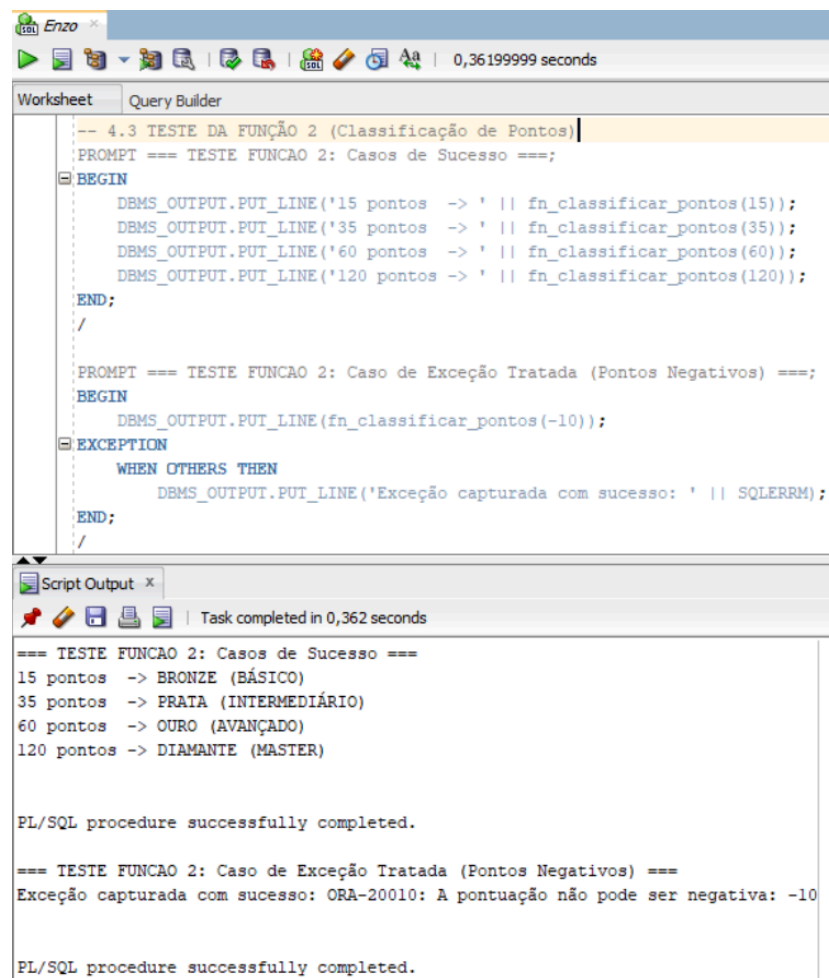
Prints



Função 2 — fn_classificar_pontos

Caso	Entrada	Resultado Esperado
Sucesso — BRONZE	15	BRONZE (BÁSICO)
Sucesso — PRATA	35	PRATA (INTERMEDIÁRIO)
Sucesso — OURO	60	OURO (AVANÇADO)
Sucesso — DIAMANTE	120	DIAMANTE (MASTER)
Exceção — negativo	-10	ORA-20010

Prints



```
-- 4.3 TESTE DA FUNÇÃO 2 (Classificação de Pontos)
PROMPT === TESTE FUNCAO 2: Casos de Sucesso ===;
BEGIN
  DBMS_OUTPUT.PUT_LINE('15 pontos -> ' || fn_classificar_pontos(15));
  DBMS_OUTPUT.PUT_LINE('35 pontos -> ' || fn_classificar_pontos(35));
  DBMS_OUTPUT.PUT_LINE('60 pontos -> ' || fn_classificar_pontos(60));
  DBMS_OUTPUT.PUT_LINE('120 pontos -> ' || fn_classificar_pontos(120));
END;
/

PROMPT === TESTE FUNCAO 2: Caso de Exceção Tratada (Pontos Negativos) ===;
BEGIN
  DBMS_OUTPUT.PUT_LINE(fn_classificar_pontos(-10));
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Exceção capturada com sucesso: ' || SQLERRM);
END;
/
```

Script Output x

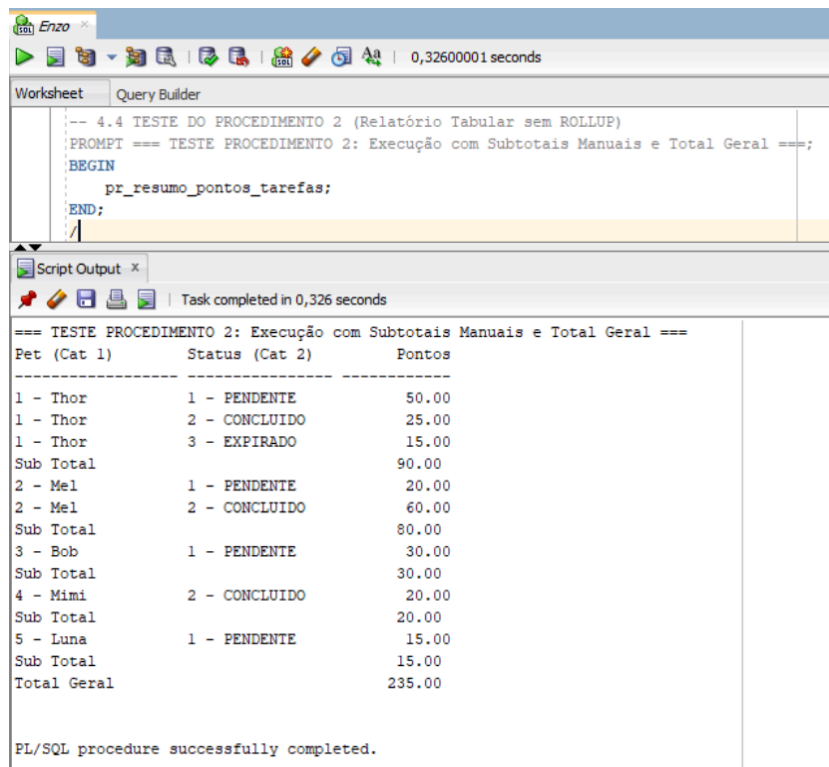
Task completed in 0,362 seconds

```
=== TESTE FUNCAO 2: Casos de Sucesso ===
15 pontos -> BRONZE (BÁSICO)
35 pontos -> PRATA (INTERMEDIÁRIO)
60 pontos -> OURO (AVANÇADO)
120 pontos -> DIAMANTE (MASTER)

PL/SQL procedure successfully completed.

=== TESTE FUNCAO 2: Caso de Exceção Tratada (Pontos Negativos) ===
Exceção capturada com sucesso: ORA-20010: A pontuação não pode ser negativa: -10

PL/SQL procedure successfully completed.
```



The screenshot shows the SQL Developer interface. The top pane displays a PL/SQL procedure named `pr_resumo_pontos_tarefas` within a block `TESTE DO PROCEDIMENTO 2`. The bottom pane shows the output of the procedure execution, which is a table with three columns: `Pet (Cat 1)`, `Status (Cat 2)`, and `Pontos`. The output lists tasks for five pets: Thor, Mel, Bob, Mimi, and Luna, with their respective status and points. The total points for all tasks is 235.00. Below the table, a message states: `PL/SQL procedure successfully completed.`

```
-- 4.4 TESTE DO PROCEDIMENTO 2 (Relatório Tabular sem ROLLUP)
PROMPT === TESTE PROCEDIMENTO 2: Execução com Subtotais Manuais e Total Geral ===;
BEGIN
    pr_resumo_pontos_tarefas;
END;
```

Pet (Cat 1)	Status (Cat 2)	Pontos
1 - Thor	1 - PENDENTE	50.00
1 - Thor	2 - CONCLUIDO	25.00
1 - Thor	3 - EXPIRADO	15.00
Sub Total		90.00
2 - Mel	1 - PENDENTE	20.00
2 - Mel	2 - CONCLUIDO	60.00
Sub Total		80.00
3 - Bob	1 - PENDENTE	30.00
Sub Total		30.00
4 - Mimi	2 - CONCLUIDO	20.00
Sub Total		20.00
5 - Luna	1 - PENDENTE	15.00
Sub Total		15.00
Total Geral		235.00

PL/SQL procedure successfully completed.

Trigger — trg_audit_tarefa

Os três eventos DML são disparados sobre um registro temporário (`id_tarefa = 99`) e os registros resultantes são consultados na tabela `auditoria_dml_tarefa`.

SQL

-- INSERT

```
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao,
criacao, prazo, conclusao, pet_id_pet, status_id_status,
usuario_id_usuario)
VALUES (99, 'Treino de Truque', 40, 'Ensinar comando rolar',
SYSTIMESTAMP, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 1, 1, 1);
```

-- UPDATE

```
UPDATE tarefa SET pontos_tarefa = 50, status_id_status = 2 WHERE
id_tarefa = 99;
```

-- DELETE

```
DELETE FROM tarefa WHERE id_tarefa = 99;
COMMIT;
```

-- Evidência

```
SELECT id_auditoria, nome_usuario, tipo_operacao,
data_hora_operacao, valoresanteriores, valoresnovos FROM
auditoria_dml_tarefa ORDER BY id_auditoria ASC;
```

Caso	Resultado Esperado
INSERT	Registro com valoresanteriores = null, valoresnovos preenchido
UPDATE	Registro com :OLD e :NEW preenchidos
DELETE	Registro com valoresanteriores preenchido, valoresnovos = null

Prints

The screenshot displays the Enzo SQL IDE interface. The top toolbar includes icons for running queries, saving, and other standard database operations. The main window is divided into two panes: 'Worksheet' and 'Query Builder'. The 'Worksheet' pane contains the following SQL script:

```
-- INSERT
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo,
conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (99, 'Treino de Truque', 40, 'Ensinar comando rolar',
SYSTIMESTAMP, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 1, 1, 1);

-- UPDATE
UPDATE tarefa SET pontos_tarefa = 50, status_id_status = 2 WHERE id_tarefa = 99;

-- DELETE
DELETE FROM tarefa WHERE id_tarefa = 99;

COMMIT;

-- Evidência
SELECT id_auditoria, nome_usuario, tipo_operacao,
data_hora_operacao, valoresanteriores, valoresnovos
FROM auditoria_dml_tarefa
ORDER BY id_auditoria ASC;
```

The bottom pane shows the 'Script Output' and 'Query Result' tabs. The 'Script Output' tab displays the following messages:

```
1 row inserted.

1 row updated.

1 row deleted.

Commit complete.

>>Query Run In:Query Result
```

Script Output		Query Result			
All Rows Fetched: 3 in 0,032 seconds					
ID...	NOME_US...	TIPO...	DATA_HORA_OPERACAO	VALORES_ANTI...	VALORES_NOVOS
1	1RM561432	INSERT	11-SEP-26 05.07.3...	(null)	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=40; STATUS=1; PET=1; USUARI...
2	2RM561432	UPDATE	11-SEP-26 05.07.3...	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=40; STATUS=1; PET=1; USUARIO=1	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=50; STATUS=2; PET=1; USUARI...
3	3RM561432	DELETE	11-SEP-26 05.07.3...	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=50; STATUS=2; PET=1; USUARIO=1	(null)